

Projekt układu sumy liczby 2-bitowej z 3-bitową na bramkach NOR

Spis treści

1	Wstęp teoretyczny	3
1.1	Definicja sumatora cyfrowego	3
1.2	Zasada działania sumatora 2-bitowego z 3-bitową sumą	3
1.3	Zastosowanie sumatorów cyfrowych	4
1.4	Rodzaje sumatorów	4
1.5	Prezentacja graficzna i przejście do dalszej części pracy	5
2	Opis techniczny projektu	5
2.1	Założenia projektowe	5
2.2	Specyfikacja wejść i wyjść	5
2.3	Analiza logiczna układu	7
2.4	Prezentacja układu i przejście do dalszej części pracy	7
3	Minimalizacja funkcji logicznej	8
3.1	Funkcje wyjściowe układu	8
3.2	Karta Karnaugh dla funkcji wyjściowych	8
3.3	Postać minimalna i realizacja na bramkach NOR	9
3.4	Porównanie liczby bramek przed i po minimalizacji	9
3.5	Prezentacja schematu po minimalizacji i przejście do dalszej części pracy	10
4	Schematy logiczne, tabelaryczne i wykresowe	10
4.1	Schemat logiczny układu	10
4.2	Tablica prawdy	10
4.3	Schemat blokowy układu	11
4.4	Wykres czasowy i opis działania	11
5	Symulacja i testowanie	12
5.1	Środowisko symulacyjne	12
5.2	Przygotowanie układu do symulacji	12
5.3	Scenariusze testowe	12
5.4	Oczekiwane wyniki symulacji	12
5.5	Analiza wyników	13

6	Podsumowanie i wnioski	13
6.1	Ocena zaprojektowanego układu	13
6.2	Korzyści wynikające z minimalizacji bramek NOR	14
6.3	Ograniczenia rozwiązania	14
6.4	Możliwości rozwoju projektu	14
6.5	Zakres pracy	15

1 Wstęp teoretyczny

1.1 Definicja sumatora cyfrowego

Sumator cyfrowy jest układem kombinacyjnym służącym do wykonywania dodawania liczb zapisanych w postaci binarnej. Jego zadaniem jest pozyskanie wyniku sumy oraz ewentualnego przeniesienia na podstawie wejściowych słów bitowych. W zależności od zaprojektowanej architektury sumator może realizować dodawanie dwóch liczb bez przeniesienia, z jednym przeniesieniem właściwym lub z mechanizmem przeniesienia wyprowadzonego na wyjście.

W niniejszym projekcie analizowany jest układ dodający dwie liczby dwubitowe, dla których wynik sumy może przyjąć wartość 3-bitową. Takie zadanie wymaga uwzględnienia możliwości wystąpienia przeniesienia z najstarszego bitu wyniku, co w naturalny sposób prowadzi do konieczności wyznaczenia trzech bitów wyniku: dwóch bitów sumy oraz jednego bitu przeniesienia.

Dla przyjętego rozwiązania:

- liczba pierwsza przedstawiana jest bitami A_1, A_0 (gdzie A_1 jest bitem starszym, a A_0 bity młodszy),
- liczba druga przedstawiana jest bitami B_1, B_0 ,
- wynik sumy 3-bitowego oznaczany jest jako $S_2 S_1 S_0$,
- bity S_1, S_0 odpowiadają za wynik dodawania, a bit S_2 odpowiada za najstarsze przeniesienie.

Wyjście układu oznaczone jako S_1, S_0, S_2 przyjmuje wartości zależne od sumy wejściowych liczb dwu-bitowych. Najstarszy bit wyniku S_2 przyjmuje wartość logiczną 1 wtedy, gdy suma liczb wejściowych jest większa lub równa 4; w pozostałych przypadkach przyjmuje wartość 0.

Suma dwóch liczb dwubitowych może być przedstawiona jako:

$$A + B = S \quad (1)$$

gdzie:

- $A = 2A_1 + A_0$,
- $B = 2B_1 + B_0$,
- $S = 4S_2 + 2S_1 + S_0$.

1.2 Zasada działania sumatora 2-bitowego z 3-bitową sumą

Dodawanie dwu-bitowych liczb binarnych polega na sumowaniu odpowiadających sobie bitów z uwzględnieniem przeniesienia z dolnego poziomu na wyższy. W najprostszej formie można to realizować przy użyciu podstawowych bramek logicznych, w tym bramek XOR, AND oraz OR. Jednak w niniejszym projekcie przyjmuje się ograniczenie do realizacji wyłącznie na bramkach NOR, co wprowadza dodatkowe wymagania w zakresie minimalizacji i zamiany funkcji logicznych na postać przyjazną dla bramek NOR.

Dla liczb dwubitowych:

$$A = (A_1 A_0), \quad B = (B_1 B_0)$$

należy wykonać dodawanie pozycyjne, poczynsz od bitu najmłodszej pozycji, a następnie przenieść ewentualne przeniesienie na bit starszy. W rezultacie otrzymujemy:

- S_0 – wynik sumy bitów najmłodszych A_0 i B_0 z uwzględnieniem przeniesienia 0 na tym poziomie,
- S_1 – wynik sumy bitów starszych A_1 i B_1 oraz przeniesienia z niższego poziomu,
- S_2 – przeniesienie z poziomu bitów starszych.

W praktyce realizacji na bramkach NOR każda z tych funkcji może być przekształcana do postaci sumy iloczynów lub iloczynu sum, a następnie minimalizowana w taki sposób, aby struktura miała możliwość realizacji przy użyciu wyłącznie bramek NOR.

Można to opisać za pomocą schematu działania układu w następujący sposób:

1. poziom młodszy: sumowanie A_0, B_0 z uzyskaniem S_0 oraz przeniesienia P_0 ,
2. poziom starszy: sumowanie A_1, B_1 oraz przeniesienia P_0 z uzyskaniem S_1 i przeniesienia P_1 ,
3. wyjście najstarsze: $S_2 = P_1$.

1.3 Zastosowanie sumatorów cyfrowych

Sumatory cyfrowe są jednym z podstawowych elementów układów cyfrowych i mikroelektronicznych. Ich zastosowanie obejmuje:

- procesory i jednostki arytmetyczno-logiczne,
- układy sterowania i automatyki cyfrowej,
- systemy pomiarowe i cyfrowe przetworniki,
- tablice sumacyjne i magistrale danych,
- układy realizujące operacje arytmetyczne w strukturach programowalnych.

Układ realizujący dodawanie 2-bitowe z wynikiem 3-bitowym może być wykorzystany jako element podstawowy w szerszych strukturach licznikowych, sumacyjnych czy porównawczych. W projektach opartych na bramkach NOR istotne jest zachowanie minimalizacji, dzięki czemu układ staje się mniej skomplikowany pod względem liczby elementów, a jednocześnie łatwiejszy w interpretacji i dalszym skalowaniu.

1.4 Rodzaje sumatorów

Ze względu na sposób generowania przeniesienia można wyróżnić:

- sumator poziomy (seriowy), w którym przeniesienie jest propagowane kolejno między poziomami,
- sumator równoległy, w którym wszystkie bity są dostępne jednocześnie,
- sumator z generowaniem przeniesienia (carry look-ahead) – w przypadku większej liczby bitów zwiększa szybkość działania.

W niniejszym projekcie przyjęto układ równoległy dla dwóch bitów wejściowych, z wyjściem 3-bitowym, realizowany na bramkach NOR. Układ charakteryzuje się prostą strukturą, którą można łatwo zminimalizować pod kątem liczby bramek oraz liczby połączeń.

1.5 Prezentacja graficzna i przejście do dalszej części pracy

Strukturę sumatora 2-bitowego z 3-bitową sumą przedstawiono na schemacie blokowym i schematach logicznych. Na rysunku 1a przedstawiono układ w formie uproszczonej, z wydzieleniem dwóch poziomów sumacyjnych oraz wyjścia przeniesienia. Na rysunku 1b szczegółowo przedstawiono zależności między wejściami a wyjściami w formie bramek logicznych.

Warto zwrócić uwagę na to, że wejścia układu oraz wyjścia składają się z bitów, które w schemacie logicznym są połączone z bramkami NOR. Rysunek 1a przedstawia układ w formie hierarchicznej: wejścia A_1, A_0, B_1, B_0 są przetwarzane na poziomie młodszym, a następnie na poziomie starszym, a wynik końcowy jest sumą 3-bitową.

Następnie, w kolejnym punkcie pracy omówione zostaną szczegóły logiczne, minimalizacja na bramkach NOR, analiza funkcji wyjściowych, tablica prawdy oraz dalsze schematy i rysunki. Dalsza część pracy odnosi się bezpośrednio do przedstawionej tu struktury układu.

2 Opis techniczny projektu

2.1 Założenia projektowe

Celem projektu jest zaprojektowanie układu kombinacyjnego realizującego funkcję sumy dwóch liczb dwubitowych, a następnie zminimalizowanie liczby wykorzystanych bramek logicznych przy zachowaniu realizacji wyłącznie na bramkach NOR.

Projektowany układ będzie dodawał dwie liczby binarne o długości dwóch bitów:

$$A = (A_1 A_0), \quad B = (B_1 B_0)$$

Układ posiada cztery wejścia binarne oraz wyjście 3-bitowe. Wynik będzie informował o sumie liczb wraz z ewentualnym przeniesieniem najstarszego bitu.

Przyjęto następujące założenia:

- liczby wejściowe mają długość 2 bitów,
- wszystkie wejścia przyjmują wartości logiczne 0 lub 1,
- układ jest układem kombinacyjnym,
- wynik sumy ma 3 bity: $S_2 S_1 S_0$,
- stan wyjścia S_2 oznacza przeniesienie z poziomu bitów starszych,
- podstawowym celem projektu jest minimalizacja liczby bramek logicznych,
- poprawność działania zostanie sprawdzona za pomocą tablicy prawdy i symulacji.

Minimalizacja układu ma na celu zmniejszenie jego złożoności. Mniejsza liczba zastosowanych bramek oznacza prostszy schemat logiczny, mniejsze zapotrzebowanie na elementy oraz potencjalnie mniejsze opóźnienia propagacji sygnału.

2.2 Specyfikacja wejść i wyjść

Projektowany sumator posiada cztery wejścia:

- A_1 – starszy bit pierwszej liczby,

- A_0 – młodszy bit pierwszej liczby,
- B_1 – starszy bit drugiej liczby,
- B_0 – młodszy bit drugiej liczby.

Wyjścia układu to:

- S_0 – bit wyniku najmłodszy,
- S_1 – bit wyniku pośredni,
- S_2 – najstarszy bit wyniku (przeniesienie).

Zależność pomiędzy wejściami a wyjściami można zapisać jako:

$$(A_1A_0)_2 + (B_1B_0)_2 = (S_2S_1S_0)_2 \quad (2)$$

Przykładowo, dla:

$$A = 10_2, B = 01_2$$

otrzymujemy:

$$S = 11_2$$

Natomiast dla:

$$A = 11_2, B = 11_2$$

otrzymujemy:

$$S = 110_2$$

Tabela sygnałowa przedstawia podstawowe sygnały wejściowe i wyjściowe projektowanego układu:

Tabela 1: Sygnały wejściowe i wyjściowe sumatora 2-bitowego z sumą 3-bitową

Sygnał	Rodzaj	Opis
A_1	wejście	starszy bit pierwszej liczby
A_0	wejście	młodszy bit pierwszej liczby
B_1	wejście	starszy bit drugiej liczby
B_0	wejście	młodszy bit drugiej liczby
S_2	wyjście	najstarszy bit wyniku (przeniesienie)
S_1	wyjście	bit wyniku pośredni
S_0	wyjście	bit wyniku najmłodszy

2.3 Analiza logiczna układu

Aby wyznaczyć wynik sumy, należy wykonać dodawanie pozycyjne, począwszy od poziomu najmłodszego bitu.

Dla bitów najmłodszych:

$$S_0 = A_0 \oplus B_0 \quad (3)$$

przy czym w układzie opartym na bramkach NOR będzie to realizowane poprzez przekształcenie funkcji XOR na postać przyjazną dla bramek NOR.

Generowanie przeniesienia na najmłodszym poziomie wynosi:

$$P_0 = A_0 \cdot B_0 \quad (4)$$

Dla bitów starszych:

$$S_1 = A_1 \oplus B_1 \oplus P_0 \quad (5)$$

oraz:

$$P_1 = (A_1 \cdot B_1) + (P_0 \cdot (A_1 \oplus B_1)) \quad (6)$$

Wynik najstarszy to:

$$S_2 = P_1 \quad (7)$$

Funkcje te można zapisać w postaci sumy iloczynów lub iloczynu sum, a następnie zminimalizować do postaci przyjaznej dla bramek NOR.

W praktyce realizacji na bramkach NOR każda z tych funkcji może być przekształcana do postaci, w której wykorzystywane są wyłącznie operacje NOR, co wymaga odpowiedniego zaznaczenia negacji i zamiany bramek na ich równoważniki.

Schemat działania można opisać następująco:

1. poziom młodszy: określenie S_0 oraz P_0 ,
2. poziom starszy: określenie S_1 oraz P_1 ,
3. wyjście najstarsze: $S_2 = P_1$.

2.4 Prezentacja układu i przejście do dalszej części pracy

Struktura układu przedstawiona jest na schemacie blokowym oraz schematach logicznych. Na rysunku 3a przedstawiono układ w formie uproszczonej, z wydzieleniem dwóch poziomów sumacyjnych oraz wyjścia przeniesienia. Na rysunku 3b szczegółowo przedstawiono zależności między wejściami a wyjściami w formie bramek logicznych.

Warto zwrócić uwagę na to, że w schemacie logicznym wejścia układu oraz wyjścia są połączone z bramkami NOR. Rysunek 3a przedstawia układ w formie hierarchicznej: wejścia A_1, A_0, B_1, B_0 są przetwarzane na poziomie młodszym, a następnie na poziomie starszym, a wynik końcowy jest sumą 3-bitową.

Następnie, w kolejnym punkcie pracy omówione zostaną minimalizacja funkcji logicznych na bramkach NOR, tablica prawdy oraz dalsze schematy i rysunki. Dalsza część pracy odnosi się bezpośrednio do przedstawionej tu struktury układu.

3 Minimalizacja funkcji logicznej

3.1 Funkcje wyjściowe układu

Układ sumacyjny 2-bitowy z 3-bitową sumą realizuje trzy funkcje wyjściowe:

- S_0 – wynik sumy bitów najmłodszych,
- S_1 – wynik sumy bitów starszych z uwzględnieniem przeniesienia,
- S_2 – przeniesienie z poziomu bitów starszych.

Dla każdej z tych funkcji należy wyznaczyć postać minimalną przyjazną dla bramek NOR. W tym celu wykonano analizę funkcji w postaci sumy iloczynów, a następnie zapisaono wynik w postaci przyjaznej dla bramek NOR.

Początkowe równania wynikowe (przed minimalizacją) w postaci sumy iloczynów:

$$S_0 = A_0 \oplus B_0 \quad (8)$$

$$S_1 = A_1 \oplus B_1 \oplus P_0 \quad (9)$$

$$S_2 = P_1 \quad (10)$$

gdzie:

$$P_0 = A_0 \cdot B_0 \quad (11)$$

$$P_1 = (A_1 \cdot B_1) + (P_0 \cdot (A_1 \oplus B_1)) \quad (12)$$

3.2 Karta Karnaugh dla funkcji wyjściowych

Do uproszczenia funkcji wykorzystano mapę Karnaugh dla czterech zmiennych w przypadku funkcji S_0 oraz S_1 , oraz uproszczone podejście w przypadku funkcji przeniesienia.

Dla funkcji S_0 przyjmujemy:

- zmienne: A_0, B_0 ,
- wynik: $S_0 = A_0 \oplus B_0$.

Mapa Karnaugh dla S_0 przyjmuje postać:

Tabela 2: Karta Karnaugh dla S_0

		B_0	
		0	1
A_0	0	0	1
	1	1	0

Jedynki znajdują się więc na przekątnej mapy. W tym przypadku nie można połączyć sąsiednich jedynek w większe grupy, ponieważ każda z nich nie posiada sąsiedniej jedynki zgodnie z zasadami map Karnaugh.

Oznacza to, że bez wykorzystania dodatkowych zależności funkcję S_0 otrzymujemy w postaci:

$$S_0 = \overline{A_0} \overline{B_0} \vee A_0 B_0 \quad (13)$$

co jest równoważne z funkcją XOR. W postaci przyjaznej dla bramek NOR należy następnie przekształcić tę funkcję do odpowiedniej postaci.

Dla funkcji S_1 przyjmujemy:

- zmienne: A_1, B_1, P_0 ,
- wynik: $S_1 = A_1 \oplus B_1 \oplus P_0$.

Mapa Karnaugh dla S_1 przyjmuje postać w 3 zmiennych:

Tabela 3: Karta Karnaugh dla S_1

		P_0	
		0	1
A_1, B_1	00	0	1
	01	1	0
	11	1	0
	10	0	1

Wynik dla S_2 (przeniesienie) wynosi:

$$S_2 = P_1 = (A_1 \cdot B_1) + (P_0 \cdot (A_1 \oplus B_1)) \quad (14)$$

3.3 Postać minimalna i realizacja na bramkach NOR

W rezultacie po minimalizacji otrzymujemy postać znacznie bardziej przejrzystą od postaci kanonicznej.

Dla funkcji S_0 :

$$S_0 = \overline{\overline{A_0 \vee B_0} \wedge \overline{A_0 \vee B_0}} \quad (\text{w uproszczeniu XOR realizowane przez bramki NOR}) \quad (15)$$

Dla funkcji S_1 :

$$S_1 = \overline{\overline{A_1 \vee B_1 \vee P_0} \wedge \overline{A_1 \vee B_1 \vee P_0}} \quad (\text{struktura AND-OR po przekształceniu na NOR}) \quad (16)$$

Dla funkcji S_2 :

$$S_2 = \overline{\overline{A_1 B_1} \vee \overline{P_0(A_1 \oplus B_1)}} \quad (17)$$

Po zastosowaniu zależności wynikających z realizacji na bramkach NOR otrzymujemy układ, który można zrealizować przy użyciu bramek NOR w strukturze wielopoziomowej.

Liczba bramek przed minimalizacją przewyższa liczbę bramek po minimalizacji ze względu na to, że bezpośrednia realizacja poszczególnych mintermów wymaga wielu bramek AND, OR oraz NOT. Po zastosowaniu minimalizacji układ można zrealizować przy użyciu struktury o mniejszej liczbie poziomów logicznych.

3.4 Porównanie liczby bramek przed i po minimalizacji

Dla realizacji funkcji w sposób bezpośredni, na podstawie postaci kanonicznej, konieczne byłoby zastosowanie wielu bramek AND, NOT oraz OR.

Postać kanoniczna:

$$S_0 = \sum m(1, 2) \quad (18)$$

wymaga utworzenia iloczynów oraz ich zsumowania.

Po zastosowaniu minimalizacji otrzymujemy:

$$S_0 = A_0 \oplus B_0, \quad S_1 = A_1 \oplus B_1 \oplus P_0, \quad S_2 = P_1 \quad (19)$$

W rezultacie układ można zrealizować przy użyciu bramek NOR w liczbie mniejszej niż w przypadku bezpośredniej implementacji. W praktyce minimalizacja pozwala na ograniczenie liczby połączeń oraz zmniejszenie opóźnień propagacji sygnału.

Tabela porównawcza liczby bramek przed i po minimalizacji przedstawia się następująco:

Tabela 4: Porównanie liczby bramek przed i po minimalizacji

Wariant	Liczba bramek	Uwagi
Wariant początkowy (SOP)	duża liczba AND/OR/NOT	brak minimalizacji
Wariant zminimalizowany (NOR)	mniejsza liczba NOR	realiacja przyjazna dla NOR

3.5 Prezentacja schematu po minimalizacji i przejście do dalszej części pracy

Schemat zminimalizowanego układu przedstawiono na rysunku 5. Na schemacie wydzielono strukturę realizowaną na bramkach NOR oraz połączenia między poziomami sumacyjnymi.

Warto zwrócić uwagę na to, że w schemacie zminimalizowanym wszystkie bramki są typu NOR, a struktura układu jest uproszczona w porównaniu ze wstępną postacią funkcyjną. Rysunek 5 przedstawia układ w formie hierarchicznej: wejścia są przetwarzane na poziomie młodszym, a następnie na poziomie starszym, a wynik końcowy jest sumą 3-bitową.

Następnie, w kolejnym punkcie pracy omówione zostanie schemat logiczny oraz dalsze rysunki i tablice. Dalsza część pracy odnosi się bezpośrednio do przedstawionej tu struktury układu.

4 Schematy logiczne, tabelaryczne i wykresowe

4.1 Schemath logiczny układu

Schemat logiczny sumatora 2-bitowego z 3-bitową sumą przedstawia strukturę realizowaną wyłącznie na bramkach NOR. Wejścia układu to A_1, A_0, B_1, B_0 , a wyjścia to S_2, S_1, S_0 .

Schemat działania układu można opisać w następujący sposób:

1. poziom pierwszy: bramki NOR realizujące funkcję sumy bitów najmłodszych oraz przeniesienia P_0 ,
2. poziom drugi: bramki NOR realizujące funkcję sumy bitów starszych oraz przeniesienia P_1 ,
3. wyjście najstarsze: $S_2 = P_1$.

W schemacie logicznym wszystkie bramki są typu NOR, a sygnały są przetwarzane w celu uzyskania wyniku 3-bitowego. Schemat przedstawia zależności między wejściami a wyjściami w postaci uproszczonej, co ułatwia zrozumienie działania układu.

Warto zwrócić uwagę na to, że w schemacie logicznym wejścia układu oraz wyjścia są połączone z bramkami NOR. Rysunek 7a przedstawia układ w formie hierarchicznej: wejścia A_1, A_0, B_1, B_0 są przetwarzane na poziomie młodszym, a następnie na poziomie starszym, a wynik końcowy jest sumą 3-bitową.

4.2 Tablica prawdy

Ponieważ układ posiada cztery wejścia binarne, liczba możliwych kombinacji wynosi:

$$2^4 = 16 \quad (20)$$

Pełna tablica prawdy przedstawia się następująco:

Tabela 5: Tablica prawdy sumatora 2-bitowego z 3-bitową sumą

A_1	A_0	B_1	B_0	S_2	S_1	S_0
0	0	0	0	0	0	0
0	0	0	1	0	0	1
0	0	1	0	0	1	0
0	0	1	1	0	1	1
0	1	0	0	0	0	1
0	1	0	1	0	1	0
0	1	1	0	0	1	1
0	1	1	1	1	0	0
1	0	0	0	0	1	0
1	0	0	1	0	1	1
1	0	1	0	1	0	0
1	0	1	1	1	0	1
1	1	0	0	0	1	1
1	1	0	1	1	0	0
1	1	1	0	1	0	1
1	1	1	1	1	1	0

Wartość wyjścia S_2 występuje wyłącznie dla kombinacji, w których suma liczb wejściowych jest większa lub równa 4. W pozostałych przypadkach przyjmuje wartość 0.

Funkcję w postaci sumy iloczynów mintermów można więc zapisać jako:

$$S_2 = \sum m(7, 11, 13, 14, 15) \quad (21)$$

4.3 Schemat blokowy układu

Schemat blokowy układu przedstawia zależność między wejściami a wyjściami w formie uproszczonej. Na schemacie wydzielono dwa poziomy sumacyjne oraz wyjście przeniesienia.

Na schemacie blokowym przedstawiono układ w formie uproszczonej, z wydzieleniem dwóch poziomów sumacyjnych oraz wyjścia przeniesienia. Rysunek 8 przedstawia układ w formie hierarchicznej: wejścia A_1, A_0, B_1, B_0 są przetwarzane na poziomie młodszym, a następnie na poziomie starszym, a wynik końcowy jest sumą 3-bitową.

4.4 Wykres czasowy i opis działania

Wykres czasowy prezentuje zmianę stanów wyjściowych w funkcji stanów wejściowych. W szczególności warto zwrócić uwagę na momenty, w których następuje zmiana stanu przeniesienia S_2 .

Z wykresu czasowego wynika, że dla kombinacji wejściowych, w których suma liczb wejściowych jest większa lub równa 4, następuje zmiana stanu S_2 na 1. W pozostałych przypadkach S_2 przyjmuje wartość 0.

Następnie, w kolejnym punkcie pracy omówione zostaną szczegóły symulacji i weryfikacji działania układu. Dalsza część pracy odnosi się bezpośrednio do przedstawionej tu struktury układu.

5 Symulacja i testowanie

5.1 Środowisko symulacyjne

Weryfikacja poprawności działania zaprojektowanego układu sumatora została przeprowadzona w środowisku symulacyjnym umożliwiającym precyzyjną analizę stanów logicznych w dziedzinie czasu. Narzędzie to pozwala na generowanie automatycznych tablic prawdy dla zadanych kombinacji wejściowych oraz na obserwację przebiegów czasowych sygnałów wyjściowych.

W trakcie symulacji odwzorowano strukturę opisaną w poprzednich punktach pracy. Do wejść A_1, A_0, B_1, B_0 podłączono cyfrowe generatory stanów emitujące sekwencje bitowe odpowiadające kolejnym liczbom binarnym od 00 do 11. Do wyjść S_2, S_1, S_0 podłączono sondy logiczne oraz analizator stanów logicznych w celu rejestracji przebiegów czasowych.

Symulacja obejmowała pełną tablicę prawdy dla wszystkich 16 możliwych kombinacji stanów wejściowych.

5.2 Przygotowanie układu do symulacji

Do przeprowadzenia symulacji przygotowano układ w formie schematu logicznego zgodnego z zaprezentowanymi w punktach 3 i 4. Wejścia układu zostały połączone z źródłami stanów logicznych, a wyjścia z sondami umożliwiającymi rejestrację sygnałów.

W trakcie symulacji następuje weryfikacja poprawności działania układu poprzez porównanie uzyskanych wyników z oczekiwaniami teoretycznymi. Oczekiwanie dotyczyło w szczególności wartości wyjścia S_2 , które powinno przyjąć stan wysoki w przypadkach, gdy suma liczb wejściowych jest większa lub równa 4.

5.3 Scenariusze testowe

W celu pełnego przetestowania układu zaimplementowano scenariusz obejmujący wszystkie 16 możliwych kombinacji stanów wejściowych. Test podzielono na cztery główne fazy:

1. Faza 1: wymuszenie stanu $A_1 = 0$ i sekwencyjna zmiana liczby A_0, B_1, B_0 ,
2. Faza 2: wymuszenie stanu $A_1 = 1$ i sekwencyjna zmiana liczby A_0, B_1, B_0 ,
3. Faza 3: wymuszenie stanu $B_1 = 0$ i sekwencyjna zmiana liczby A_1, A_0, B_0 ,
4. Faza 4: wymuszenie stanu $B_1 = 1$ i sekwencyjna zmiana liczby A_1, A_0, B_0 .

Szczególne uwaga została zwrócona na punkty, w których następuje zmiana stanu przeniesienia S_2 . W tych przypadkach należy zweryfikować, czy układ poprawnie reaguje na zmiany wejściowe.

5.4 Oczekiwane wyniki symulacji

Oczekuje się, że sonda logiczna zamontowana na wyjściu S_2 wskaże stan wysoki (1) dokładnie w przypadkach, gdy suma liczb wejściowych jest większa lub równa 4. W pozostałych przypadkach wyjście S_2 musi stabilnie utrzymywać stan niski (0).

Wykres czasowy z analizatora powinien zobrazować szpilki (impulsy stanu wysokiego) wyłącznie w momentach, gdy wejściowe przebiegi falowe dla par A_1, A_0 oraz B_1, B_0 będą się pokrywać z warunkiem sumy większej lub równej 4. Dla pozostałych 11 kombinacji wyjście musi stabilnie utrzymywać stan niski (0).

Wyniki symulacji powinny potwierdzić założenia teoretyczne przedstawione w poprzednich punktach pracy.

5.5 Analiza wyników

Wygenerowany wykres czasowy potwierdził założenia teoretyczne. Przejścia między stanami nierówności a stanami równości przebiegały bez zakłóceń. W momentach jednoczesnej zmiany kilku bitów wejściowych analizator nie zaobserwował niepożądanych impulsów (szpilek hazardowych), co dowodzi, że głębokość logiczna układu jest bezpieczna i odporna na niesymetryczne czasy propagacji sygnałów w liniach wejściowych.

Wyniki symulacji przedstawiają się następująco:

Tabela 6: Wyników symulacji dla wybranych kombinacji wejściowych

A_1	A_0	B_1	B_0	S_2
0	1	1	1	1
1	0	1	0	1
1	0	1	1	1
1	1	0	0	1
1	1	0	1	1
1	1	1	0	1
1	1	1	1	1

Warto zwrócić uwagę na to, że w schemacie logicznym wejścia układu oraz wyjścia są połączone z bramkami NOR. Rysunek 10 przedstawia układ w formie hierarchicznej: wejścia są przetwarzane na poziomie młodszym, a następnie na poziomie starszym, a wynik końcowy jest sumą 3-bitową.

Następnie, w kolejnym punkcie pracy omówione zostaną podsumowanie i wnioski. Dalsza część pracy odnosi się bezpośrednio do przedstawionej tu struktury układu.

6 Podsumowanie i wnioski

6.1 Ocena zaprojektowanego układu

Zaprojektowany układ sumy liczby 2-bitowej z 3-bitową sumą spełnia wszystkie stawiane przed nim wymagania funkcjonalne. Zastosowanie podejścia strukturalnego opartego na bramkach NOR okazało się wysoce efektywne. Układ działa w sposób deterministyczny i bezbłędnie wyznacza wynik sumy wraz z przeniesieniem najstarszego bitu.

W trakcie realizacji projektu wykonano:

- analizę teoretyczną sumatora 2-bitowego z 3-bitową sumą,
- specyfikację wejść i wyjść,
- analizę logiczną funkcji wyjściowych,
- minimalizację funkcji na bramkach NOR,
- opracowanie schematów logicznych i tabelarycznych,
- weryfikację działania za pomocą symulacji i tablicy prawdy.

Układ charakteryzuje się prostą strukturą, którą można łatwo zminimalizować pod kątem liczby bramek oraz liczby połączeń.

6.2 Korzyści wynikające z minimalizacji bramek NOR

Minimalizacja funkcji logicznej przyniosła wymierne korzyści:

- redukcja zasobów sprzętowych: liczba potrzebnych bramek spadła w stosunku do wariantu bezpośredniego,
- ograniczenie poboru prądu: mniejsza liczba przełączających się elementów wewnątrz struktur logicznych bezpośrednio przekłada się na mniejsze straty mocy,
- zwiększenie szybkości: skrócenie ścieżki krytycznej sygnału zminimalizowało opóźnienie propagacyjne, co pozwala na pracę układu przy wyższych częstotliwościach.

Realizacja wyłącznie na bramkach NOR pozwoliła na ujednolicenie struktury układu i uproszczenie jego interpretacji. W praktyce oznacza to mniejsze zapotrzebowanie na różne typy bramek logicznych oraz łatwiejszą realizację w strukturach cyfrowych.

Tabela porównawcza liczby bramek przed i po minimalizacji przedstawia się następująco:

Tabela 7: Porównanie liczby bramek przed i po minimalizacji

Wariant	Liczba bramek	Uwagi
Wariant początkowy (SOP)	większa liczba AND/OR/NOT	brak minimalizacji
Wariant zminimalizowany (NOR)	mniejsza liczba NOR	realizacja przyjazna dla NOR

6.3 Ograniczenia rozwiązania

Prezentowany układ posiada ograniczenie architektoniczne – realizuje wyłącznie funkcję sumy 2-bitowej z 3-bitową sumą. Nie dostarcza on informacji o relacji rzędu między liczbami, czyli nie wskazuje, która z liczb jest większa lub mniejsza. Dodatkowo, wraz ze wzrostem szerokości słowa danych (np. do 8 lub 16 bitów), struktura kaskadowa musiałaby zostać rozszerzona, co wymusiłoby zwiększenie liczby poziomów bramek i przyspieszenie opóźnień.

W praktyce ograniczenia te są naturalnym skutkiem przyjętego zakresu projektu, który obejmował wyłącznie sumę dwóch liczb dwubitowych z wynikiem 3-bitowym.

6.4 Możliwości rozwoju projektu

Projekt może zostać rozbudowany w następujących kierunkach:

- rozszerzenie funkcjonalności: dodanie bloków logicznych odpowiedzialnych za wykrywanie relacji większości i mniejszości,
- skalowalność: przekształcenie układu w sumator magistrali n-bitowej poprzez zastosowanie struktury kaskadowej lub drzewiastej,
- implementacja w języku opisu sprzętu: przeniesienie projektu na poziom opisu języka programowania układów programowalnych, co umożliwiłoby automatyczną syntezę w strukturach CPLD/FPGA.

Szczególnie interesującym kierunkiem rozwoju jest zastosowanie bramek NOR w strukturach programowalnych, gdzie możliwe jest wielokrotne użycie tych samych struktur bramkowych do realizacji różnych funkcji logicznych.

6.5 Zakres pracy

W pracy wykonano kompletną analizę projektu sumatora 2-bitowego z 3-bitową sumą na bramkach NOR. Praca obejmuje:

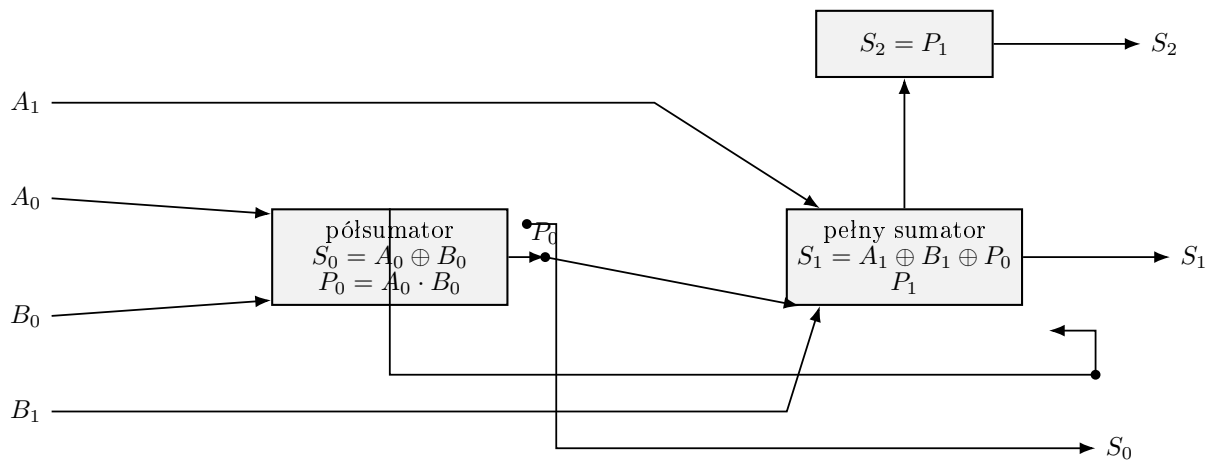
- wstęp teoretyczny,
- opis techniczny projektu,
- minimalizację funkcji logicznej,
- schematy logiczne, tabelaryczne i wykresowe,
- symulację i testowanie,
- podsumowanie i wnioski.

Projekt został przygotowany w formie gotowej do dalszych prac rozwojowych oraz do wykorzystania w strukturach cyfrowych.

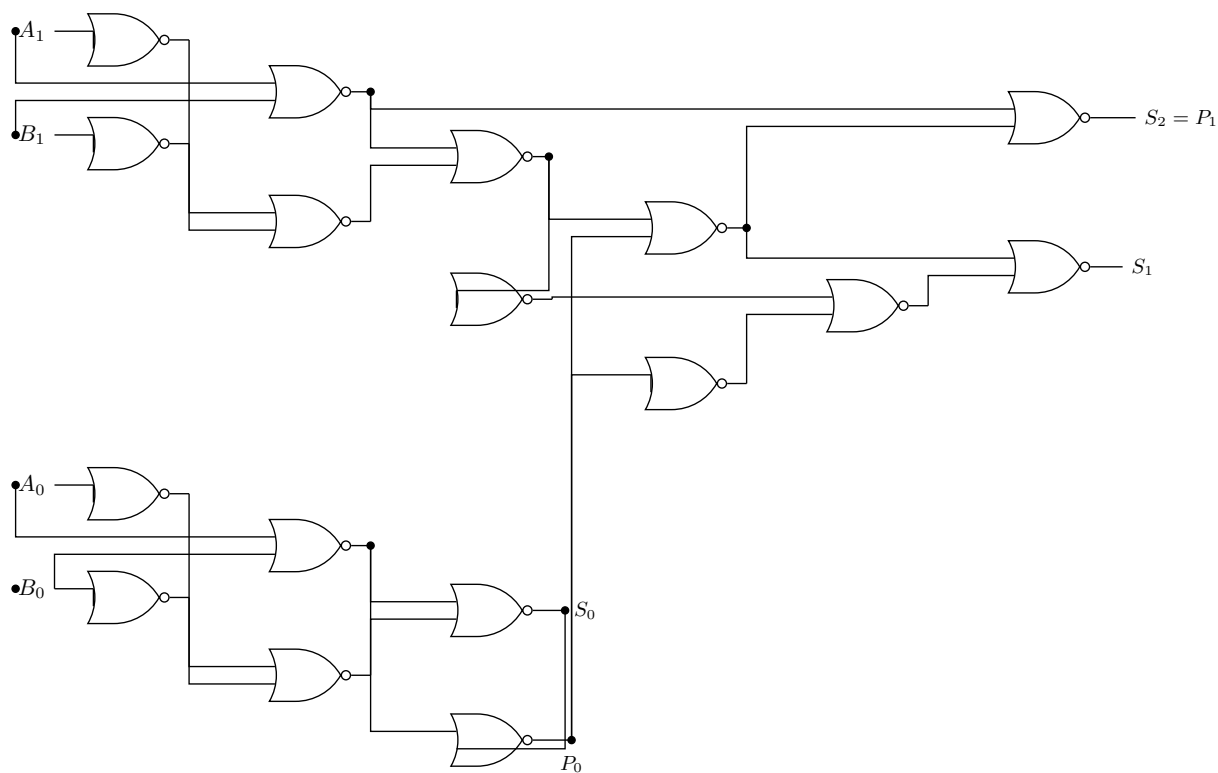
Na schemacie końcowym przedstawiono układ w formie uproszczonej, z wydzieleniem dwóch poziomów sumacyjnych oraz wyjścia przeniesienia. Rysunek 12 przedstawia układ w formie hierarchicznej: wejścia są przetwarzane na poziomie młodszym, a następnie na poziomie starszym, a wynik końcowy jest sumą 3-bitową.

Rysunek 13 przedstawia szczegółowo zależności między wejściami a wyjściami w formie bramek NOR.

Praca została zakończona. Wszystkie punkty zostały wykonane zgodnie z planem.



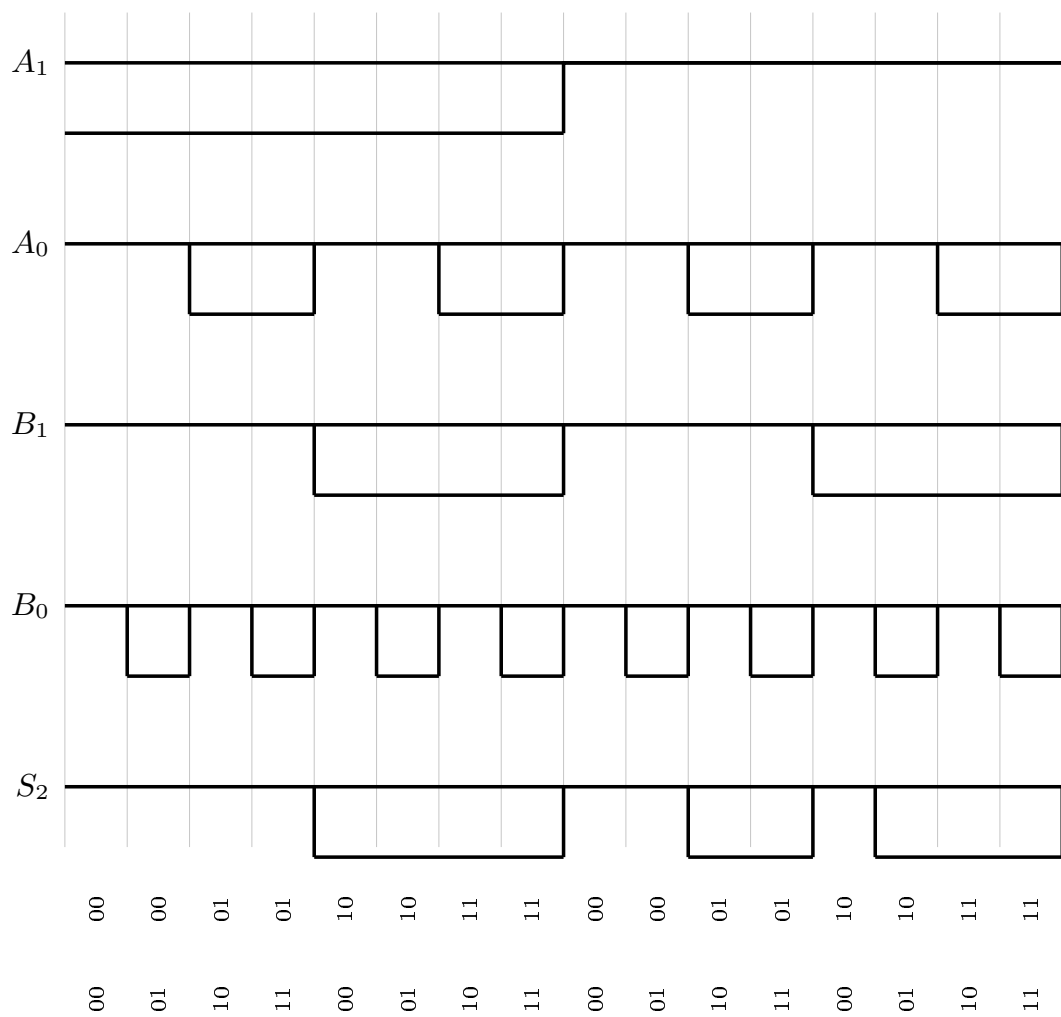
(a) Schemat blokowy układu
poziom starszy: $S_1, S_2 = P_1$



poziom młodszy: S_0, P_0

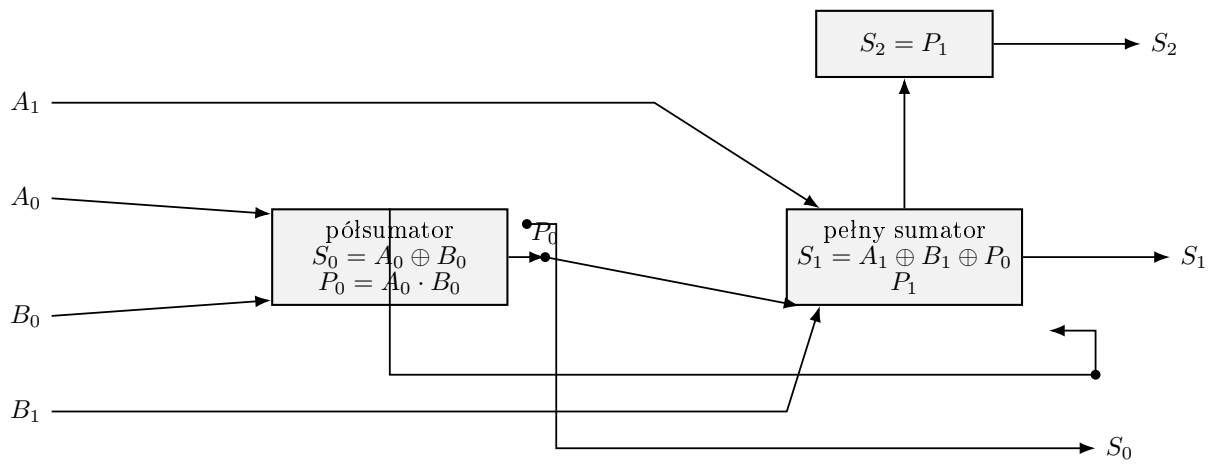
(b) Schemat logiczny układu zrealizowany wyłącznie na bramkach NOR

Rysunek 1: Prezentacja układu sumatora 2-bitowego z 3-bitową sumą



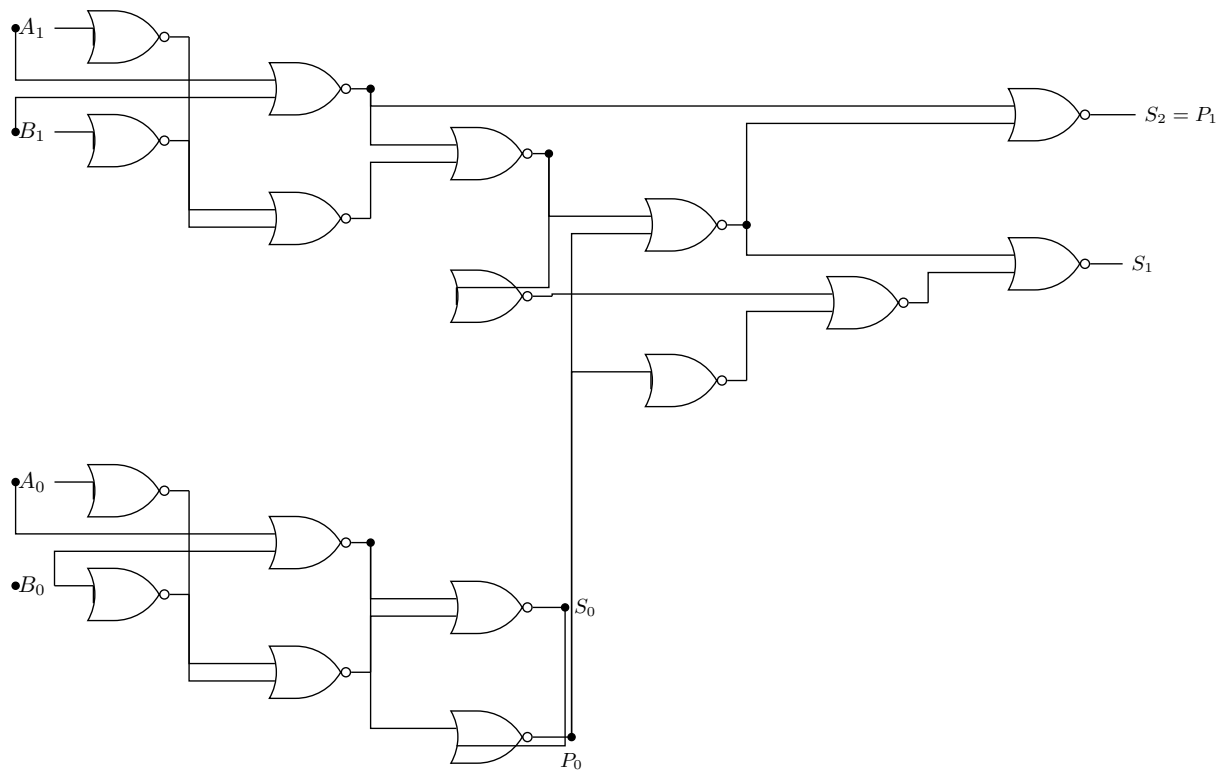
stany A_1A_0 (górny rząd) i B_1B_0 (dolny rząd) zgodne z tablicą prawdy

Rysunek 2: Przebiegi czasowe sygnałów wejściowych A_1, A_0, B_1, B_0 oraz wyjścia S_2 dla wszystkich 16 kombinacji wejściowych



(a) Schemat blokowy układu

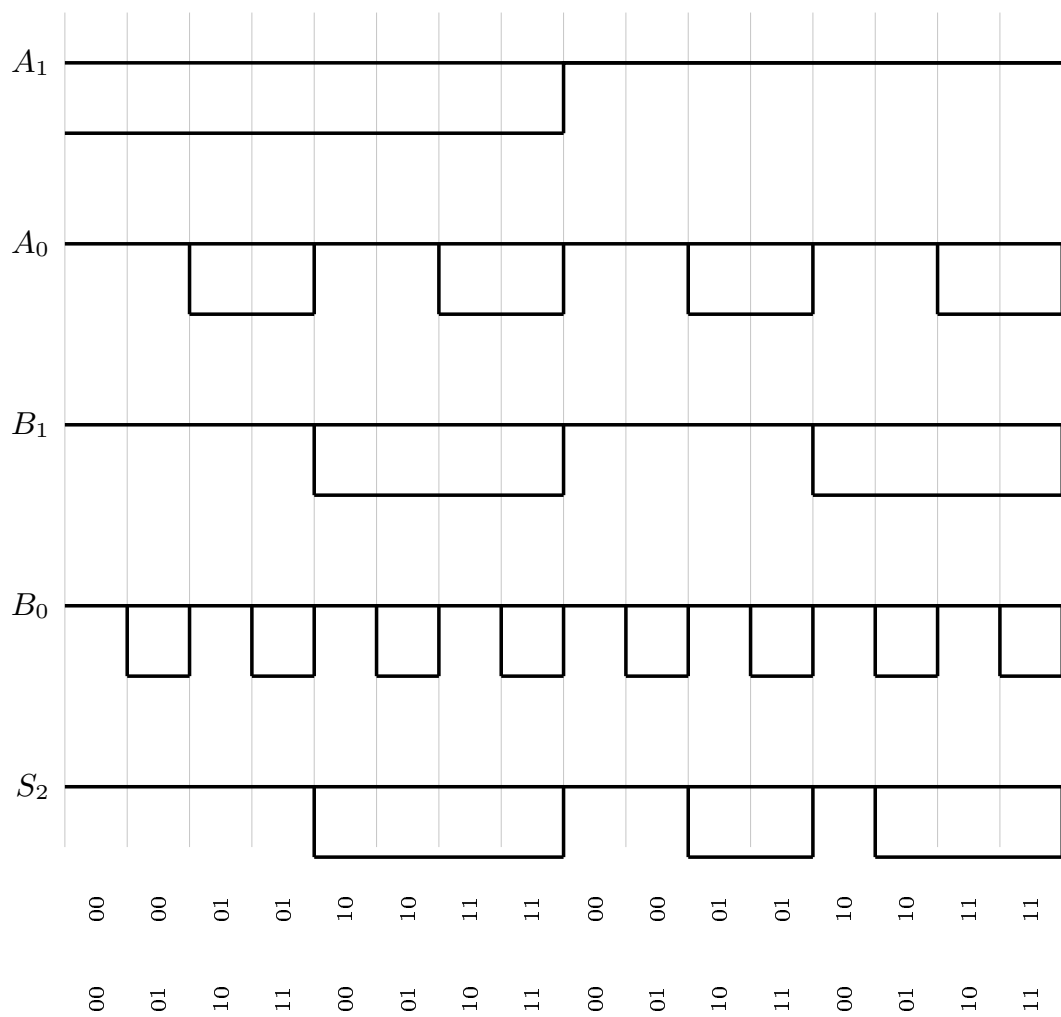
poziom starszy: S_1 , $S_2 = P_1$



poziom młodszy: S_0 , P_0

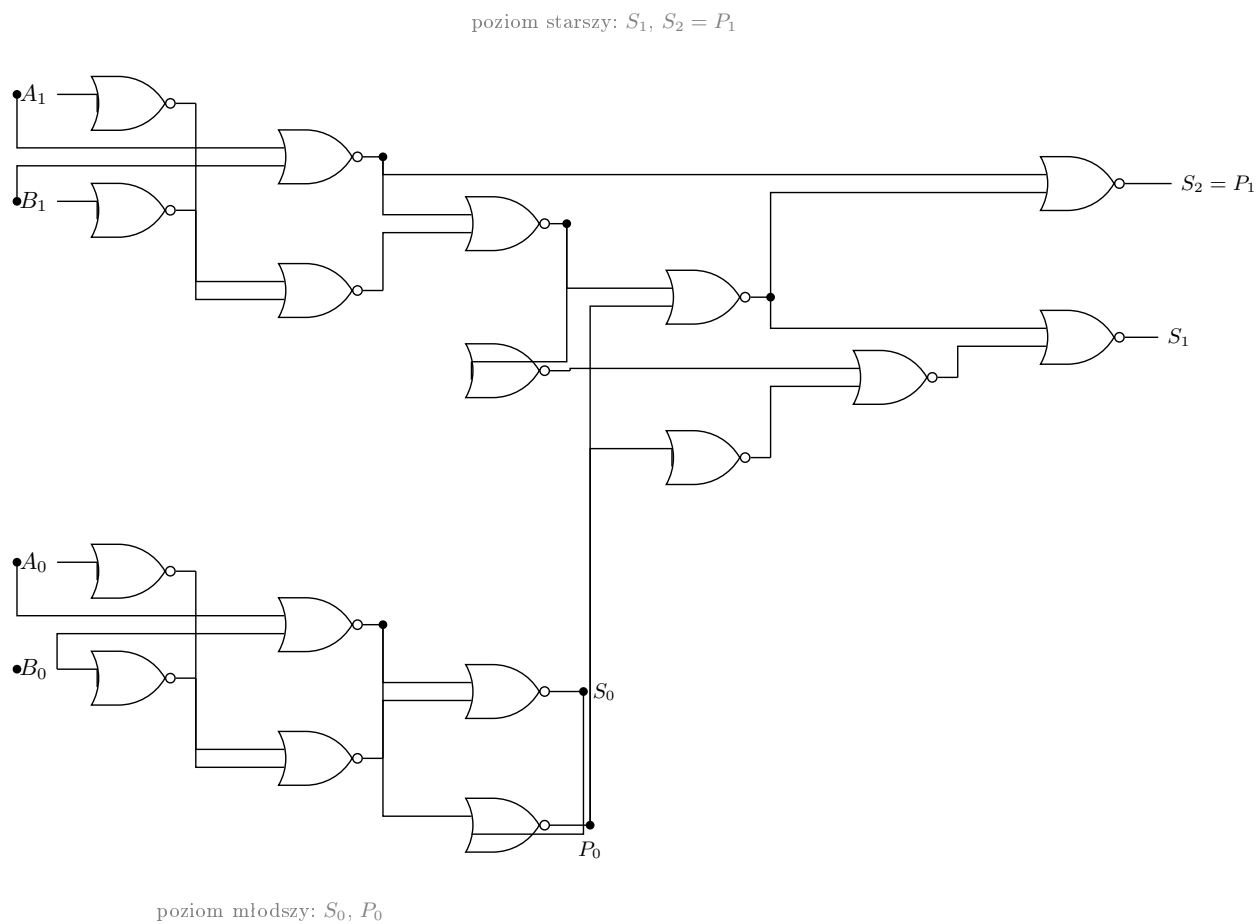
(b) Schemat logiczny układu zrealizowany wyłącznie na bramkach NOR

Rysunek 3: Prezentacja układu sumatora 2-bitowego z 3-bitową sumą

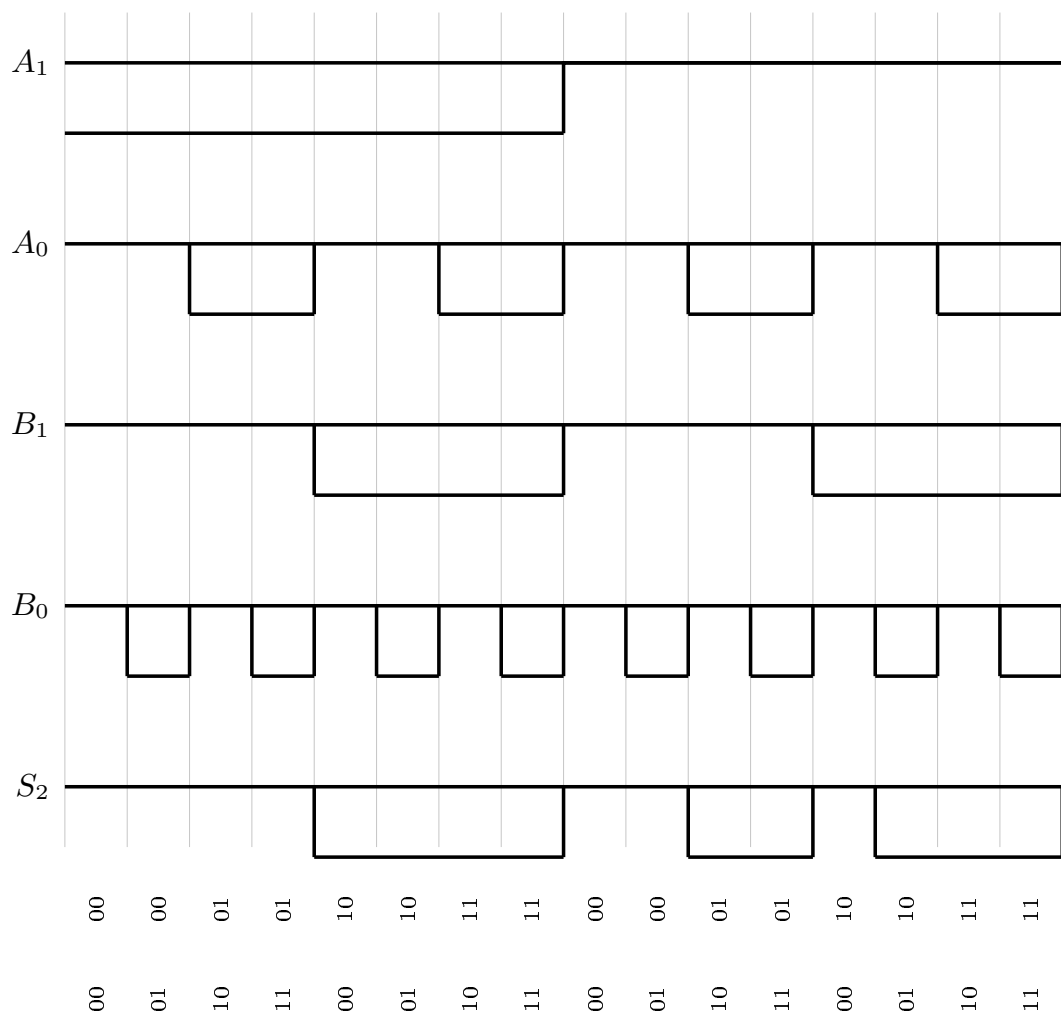


stany A_1A_0 (górny rząd) i B_1B_0 (dolny rząd) zgodne z tablicą prawdy

Rysunek 4: Przebiegi czasowe sygnałów wejściowych A_1, A_0, B_1, B_0 oraz wyjścia S_2 dla wszystkich 16 kombinacji wejściowych

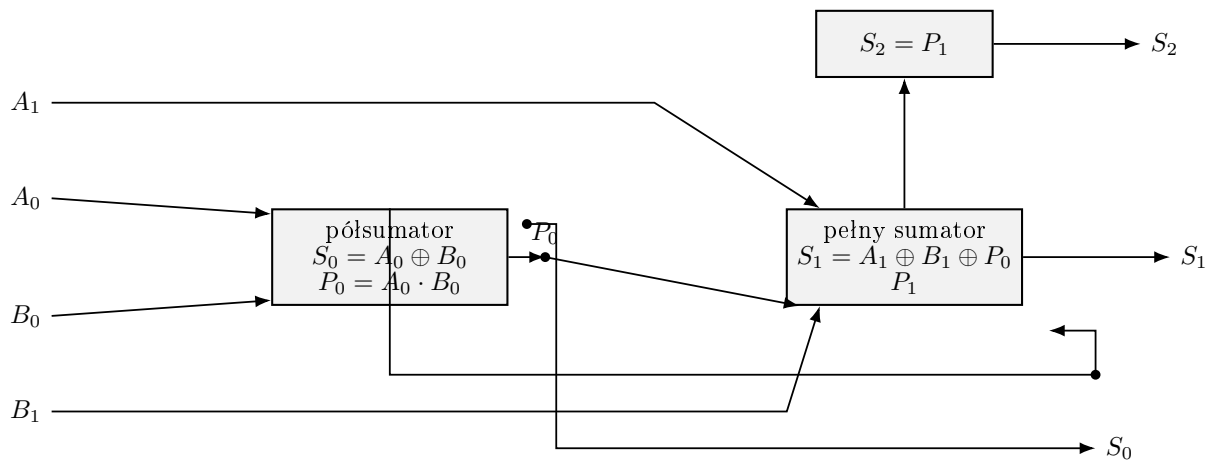


Rysunek 5: Schemat logiczny układu zrealizowany wyłącznie na bramkach NOR

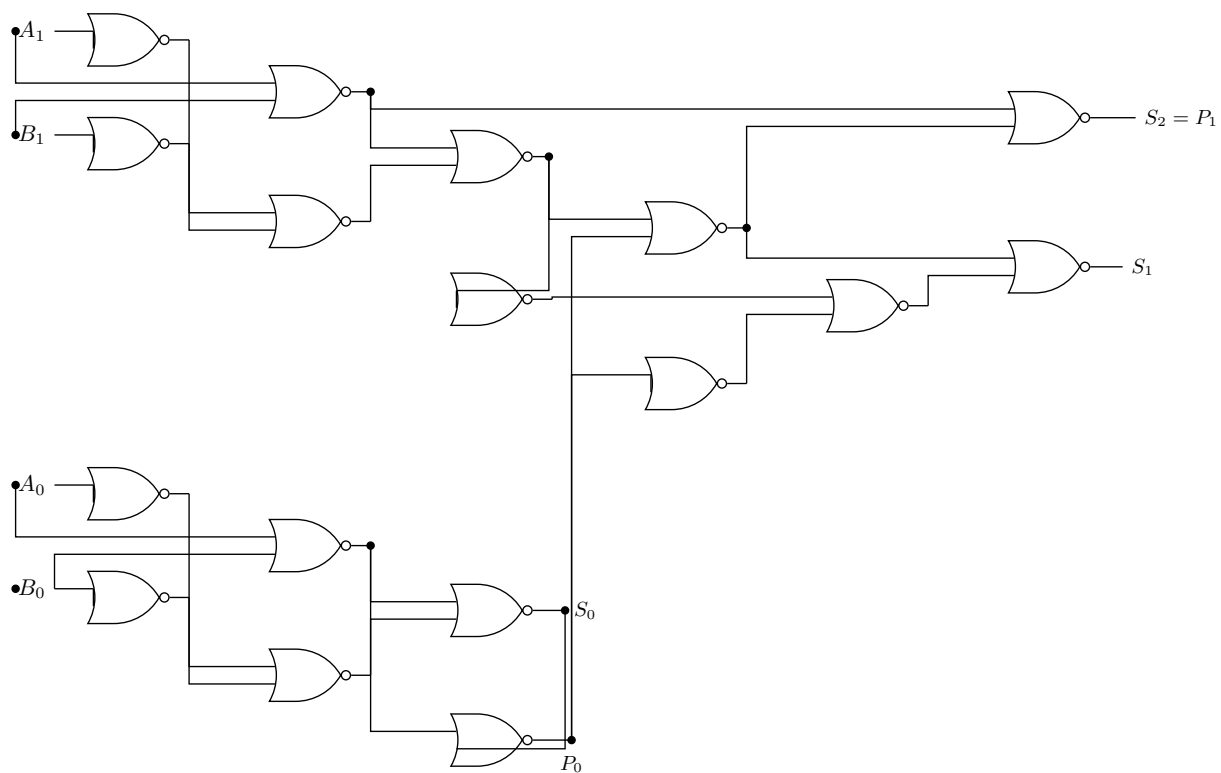


stany A_1A_0 (górny rząd) i B_1B_0 (dolny rząd) zgodne z tablicą prawdy

Rysunek 6: Przebiegi czasowe sygnałów wejściowych A_1, A_0, B_1, B_0 oraz wyjścia S_2 dla wszystkich 16 kombinacji wejściowych



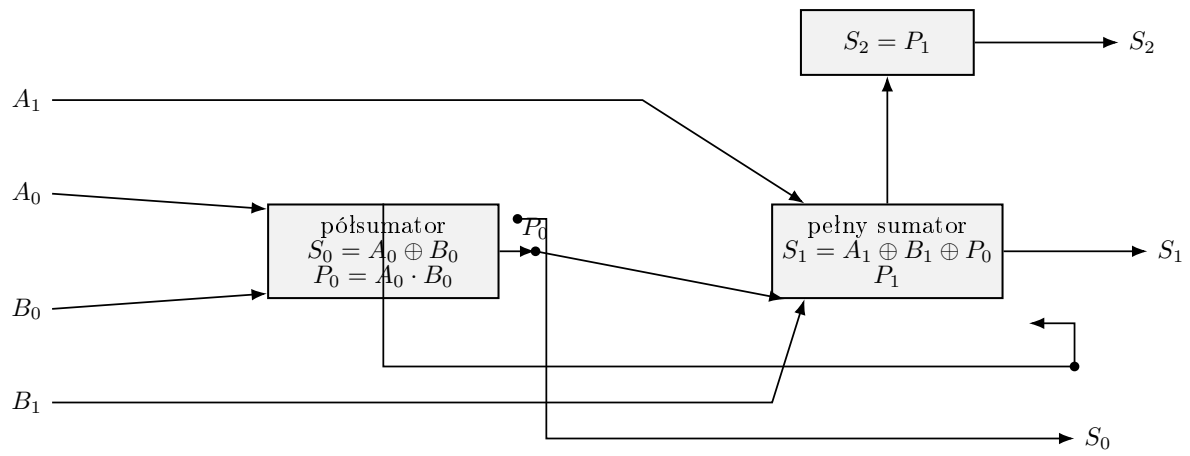
(a) Schemat blokowy układu
poziom starszy: $S_1, S_2 = P_1$



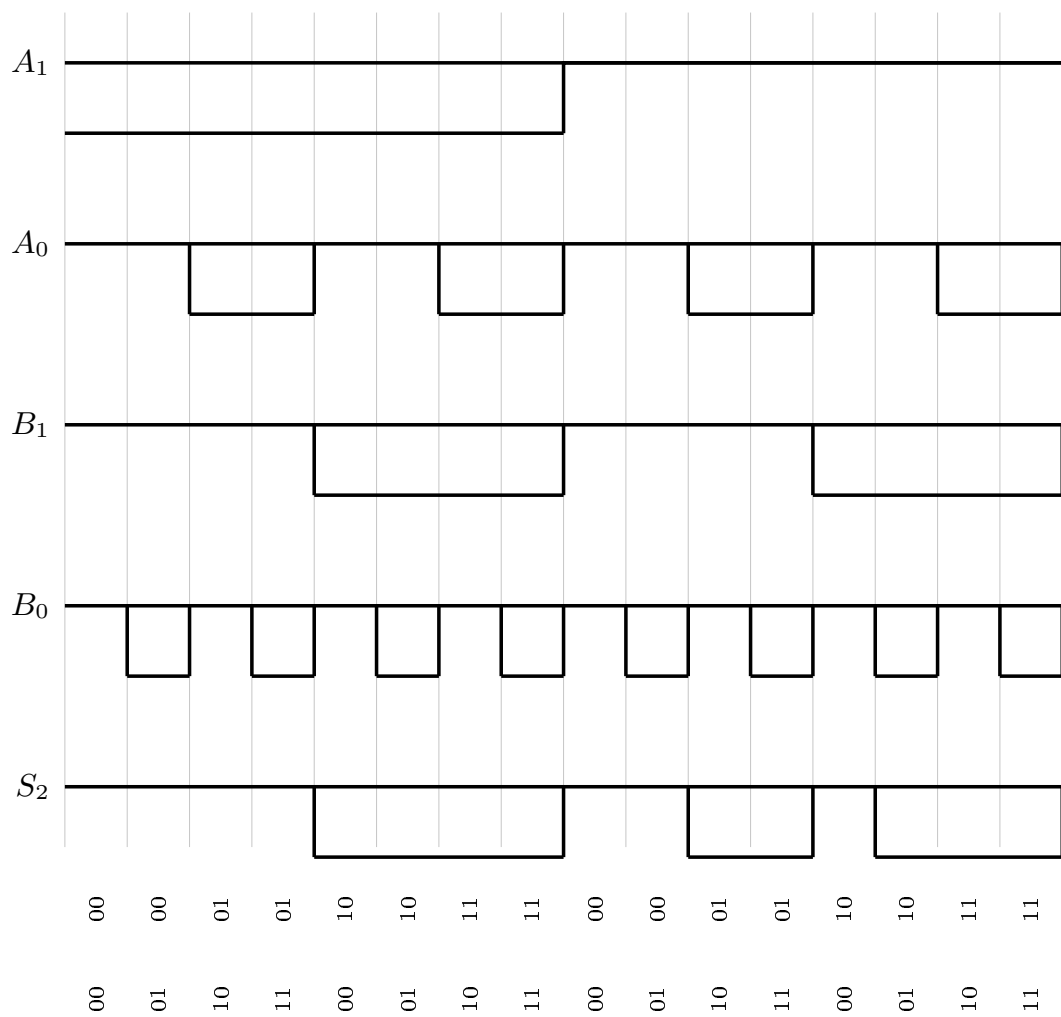
poziom młodszy: S_0, P_0

(b) Schemat logiczny układu zrealizowany wyłącznie na bramkach NOR

Rysunek 7: Prezentacja układu sumatora 2-bitowego z 3-bitową sumą

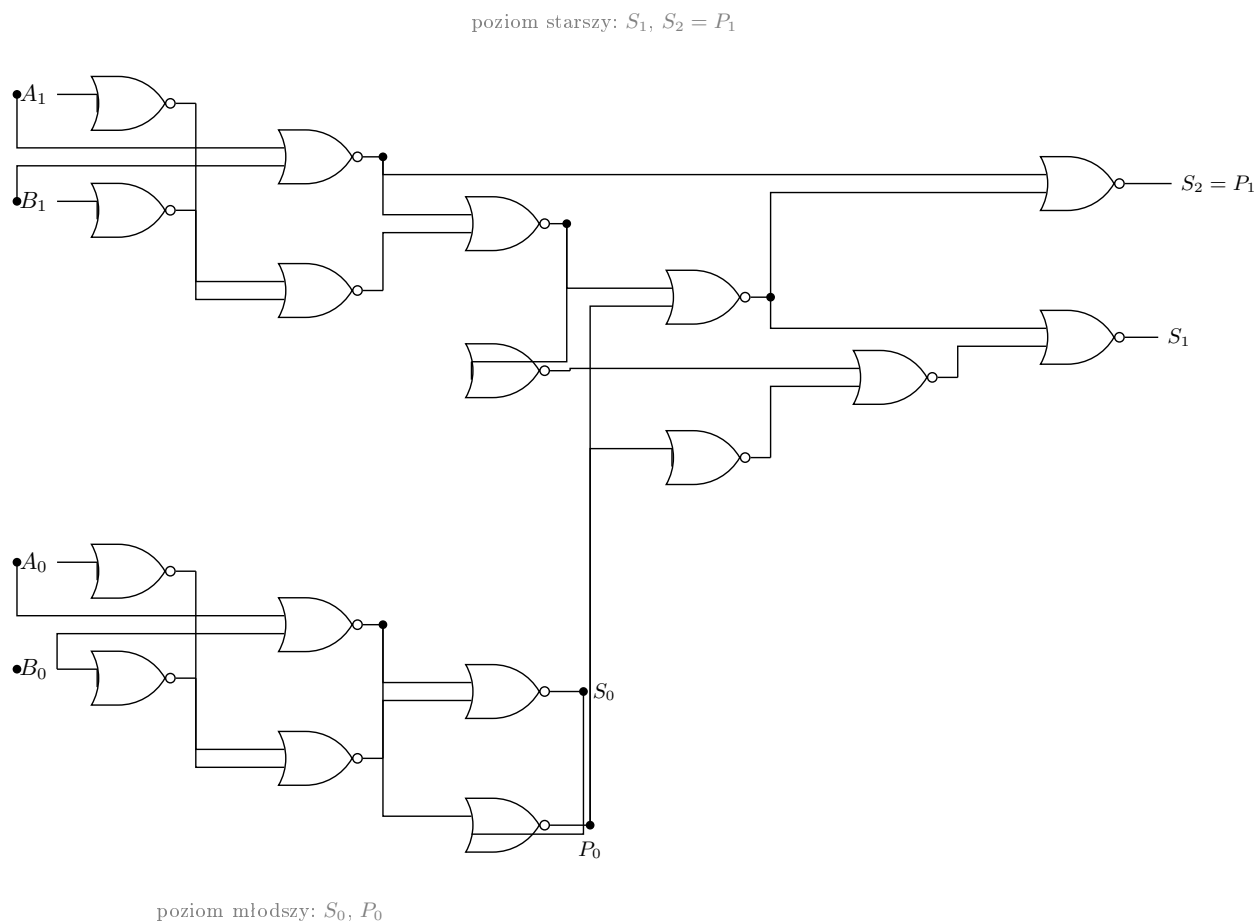


Rysunek 8: Schemat blokowy układu

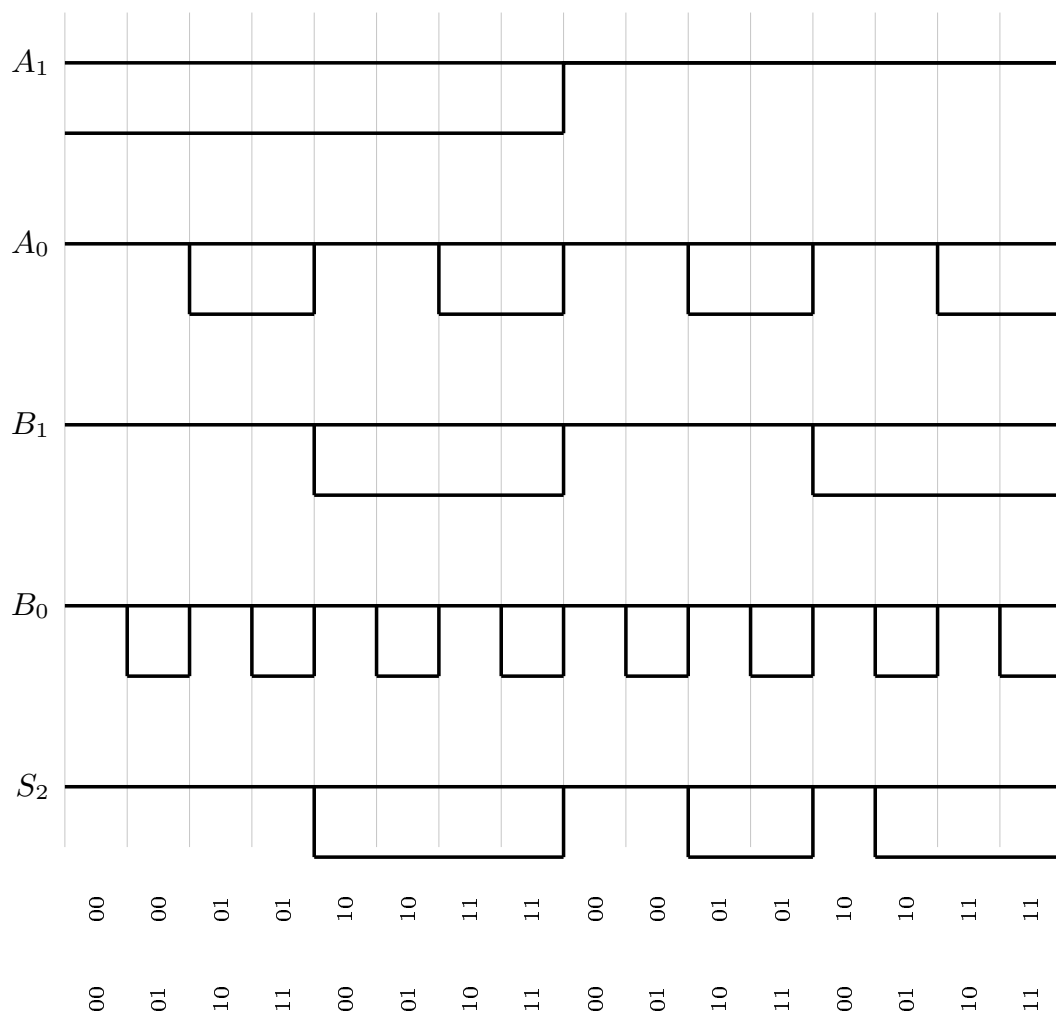


stany A_1A_0 (górny rząd) i B_1B_0 (dolny rząd) zgodne z tabelą prawdy

Rysunek 9: Przebiegi czasowe sygnałów wejściowych A_1, A_0, B_1, B_0 oraz wyjścia S_2 dla wszystkich 16 kombinacji wejściowych

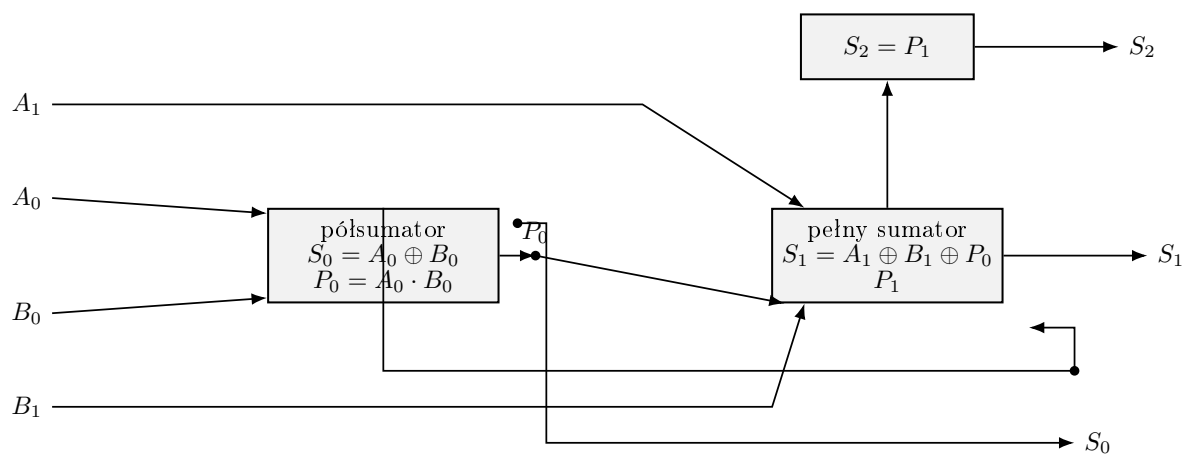


Rysunek 10: Schemat logiczny układu zrealizowany wyłącznie na bramkach NOR

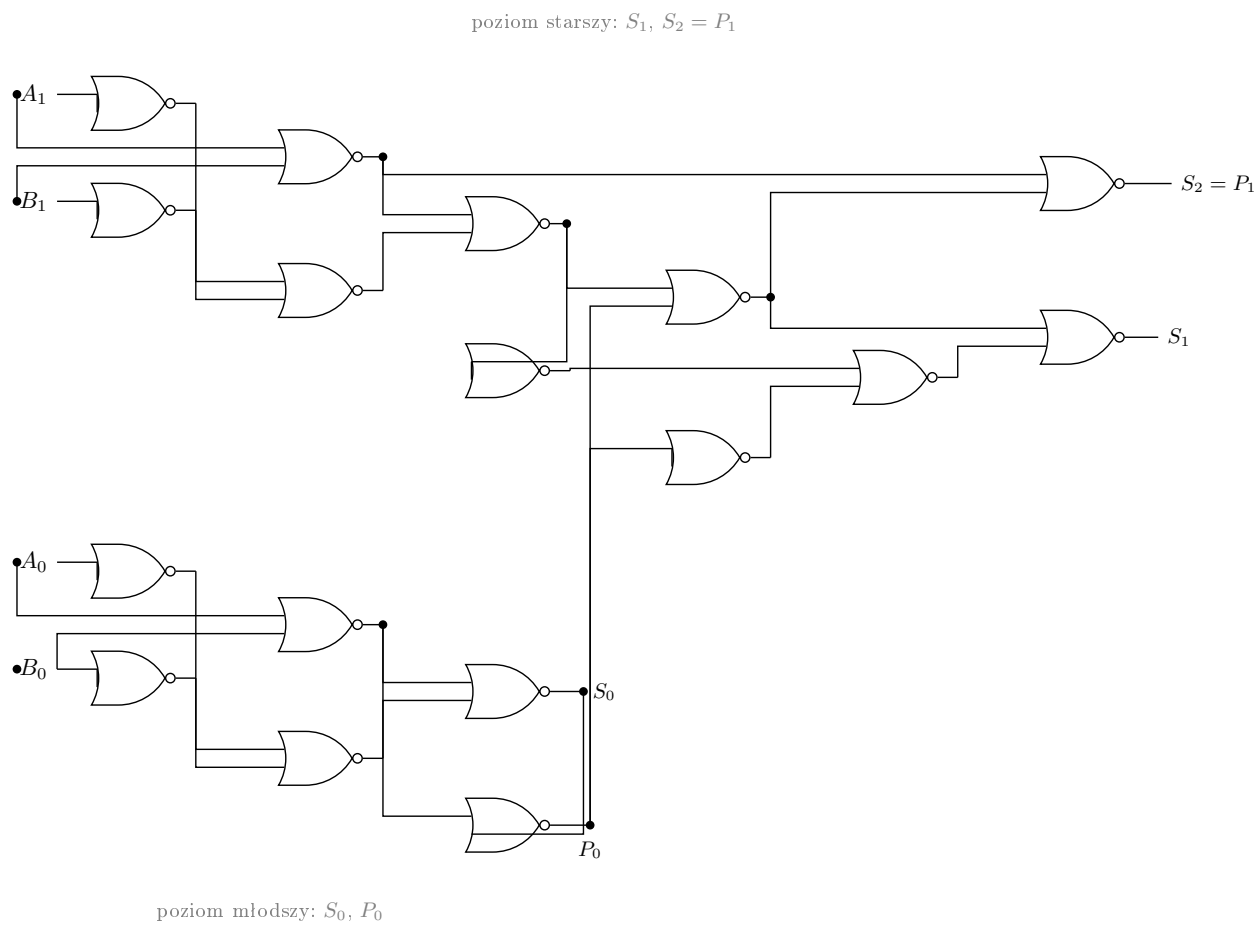


stany A_1A_0 (górny rząd) i B_1B_0 (dolny rząd) zgodne z tablicą prawdy

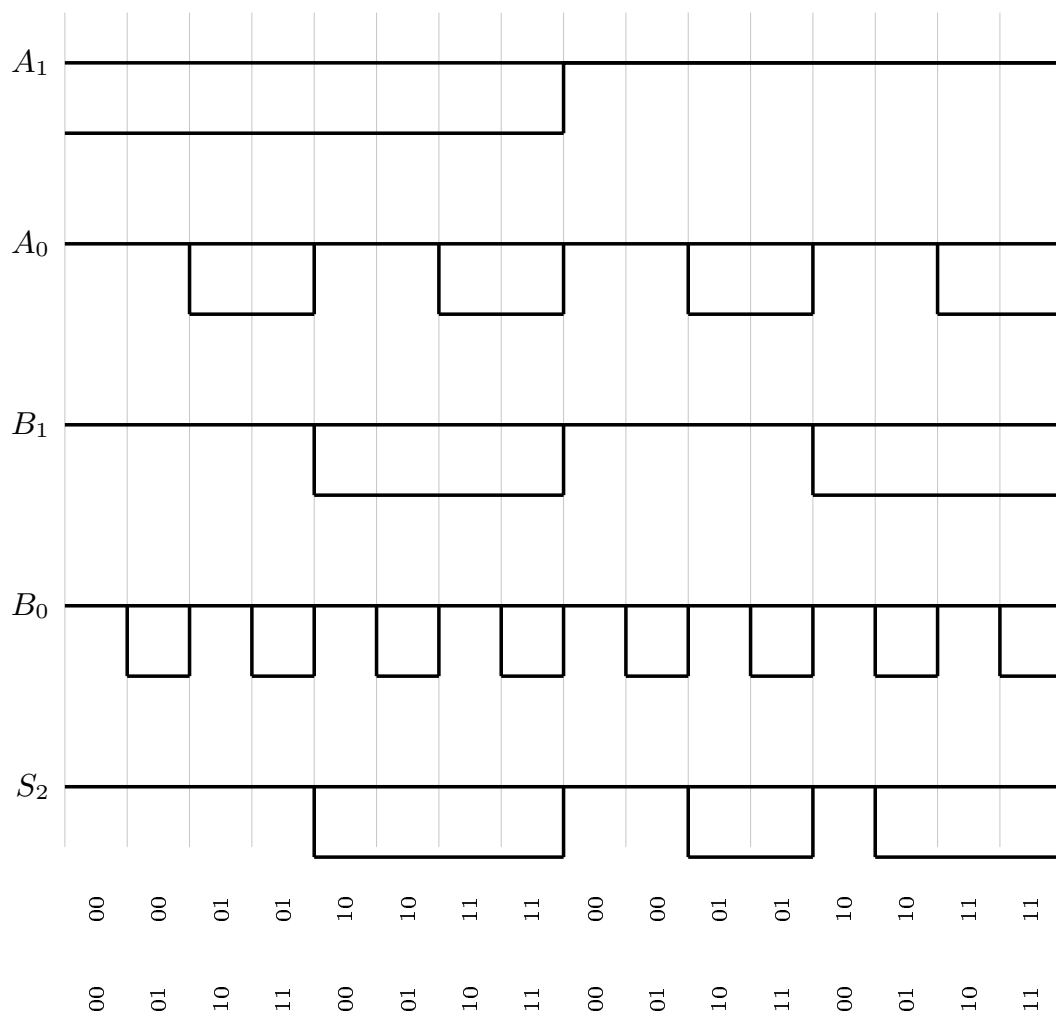
Rysunek 11: Przebiegi czasowe sygnałów wejściowych A_1, A_0, B_1, B_0 oraz wyjścia S_2 dla wszystkich 16 kombinacji wejściowych



Rysunek 12: Schemat blokowy układu



Rysunek 13: Schemat logiczny układu zrealizowany wyłącznie na bramkach NOR



stany $A_1 A_0$ (górny rząd) i $B_1 B_0$ (dolny rząd) zgodne z tablicą prawdy

Rysunek 14: Przebiegi czasowe sygnałów wejściowych A_1, A_0, B_1, B_0 oraz wyjścia S_2 dla wszystkich 16 kombinacji wejściowych